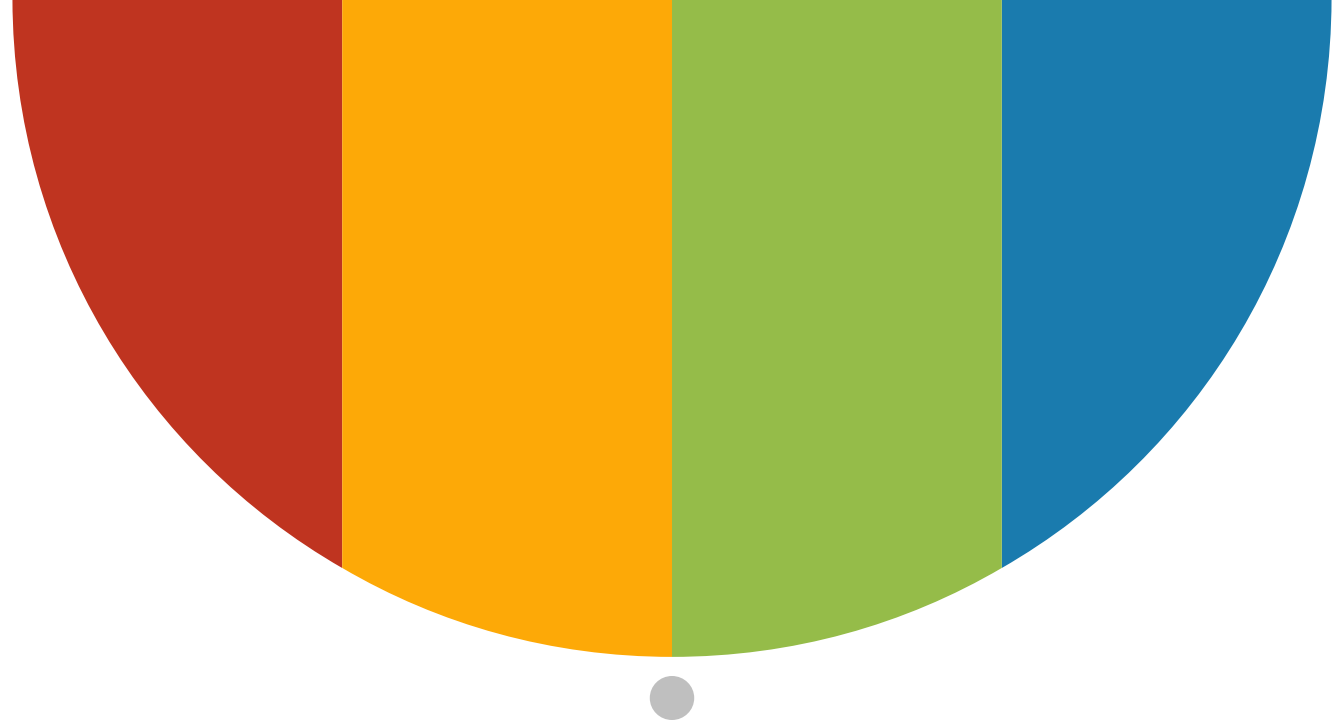




# 深度学习 Deep Learning

南开大学  
孙羽菲

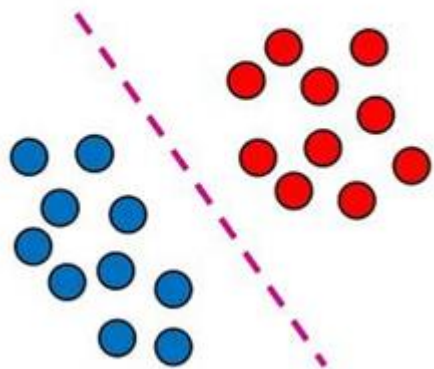
# 生成模型

Generative Models

自编码器、生成对抗网络

# 生成模型和判别模型

Discriminative



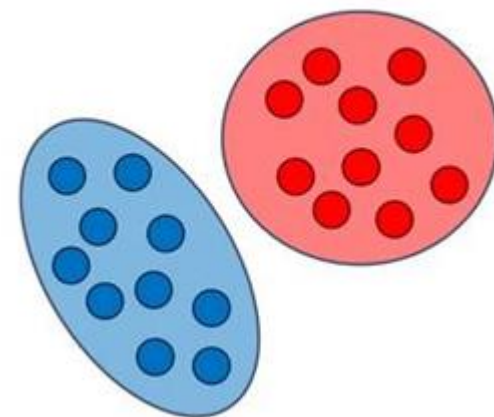
**判别模型**

学习输入数据和输出标签之间的关系，可以预测、分类

学习输入数据的分布，  
可以生成新的数据样本

**生成模型**

Generative



# 自动编码器 - AutoEncoder

自编码器就是使得**输出值近似等于输入值**的神经网络，它先将输入**编码/压缩**成潜在空间表征，然后将这种压缩后的空间表征**解码/重构**为输出。

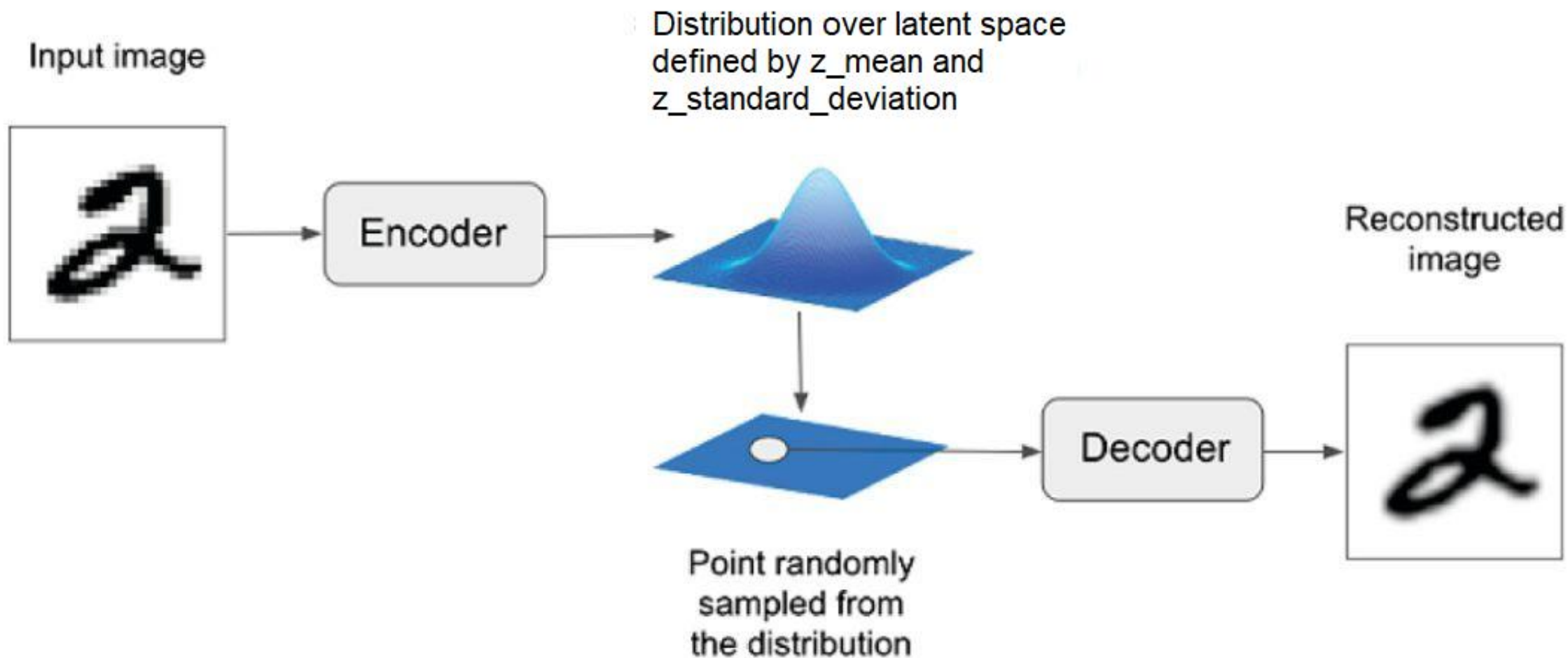


$$L(X, \hat{X}) = ||X - \hat{X}||^2$$

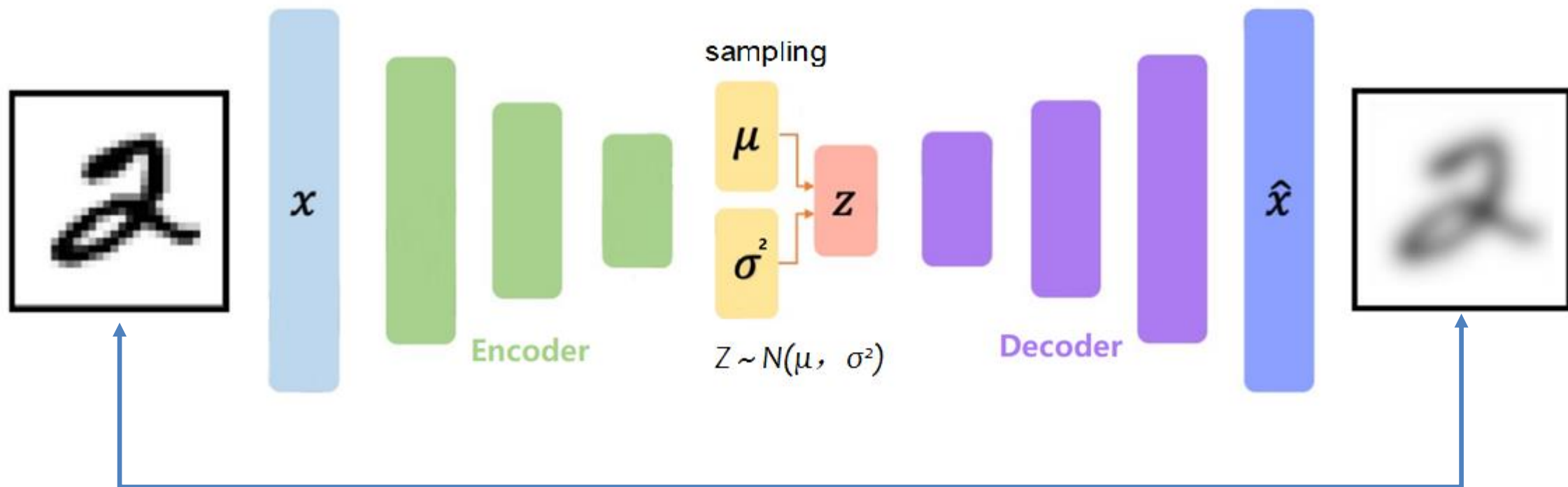
# 变分自动编码(VAE)

# 变分自动编码 (VAE)

不是将输入编码为隐空间中的单个点，而是将其编码为隐空间中的概率分布



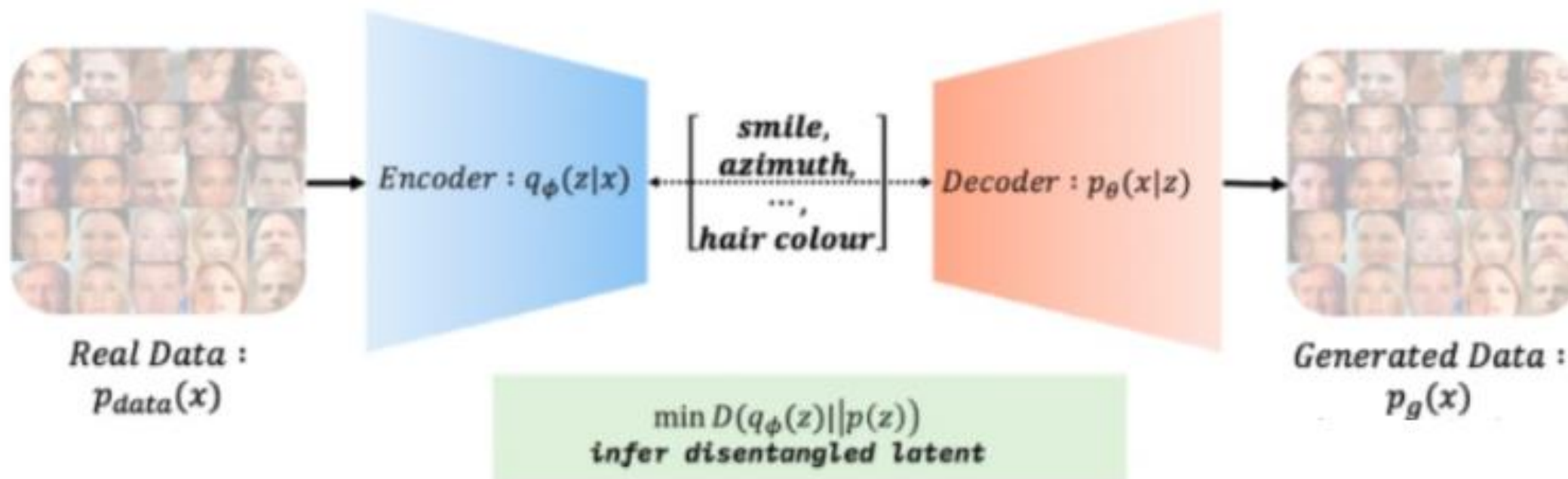
# 变分自动编码 (VAE)



$$Loss = ( \|x - \hat{x}\|^2 ) + KL( N(\mu, \sigma^2), N(1, 0) )$$

# $\beta$ -VAE

$\beta$ -VAE 是一种变分自编码器 (Variational Autoencoder) 的变体, 通过引入一个超参数  $\beta$  来控制潜在空间的解耦性。



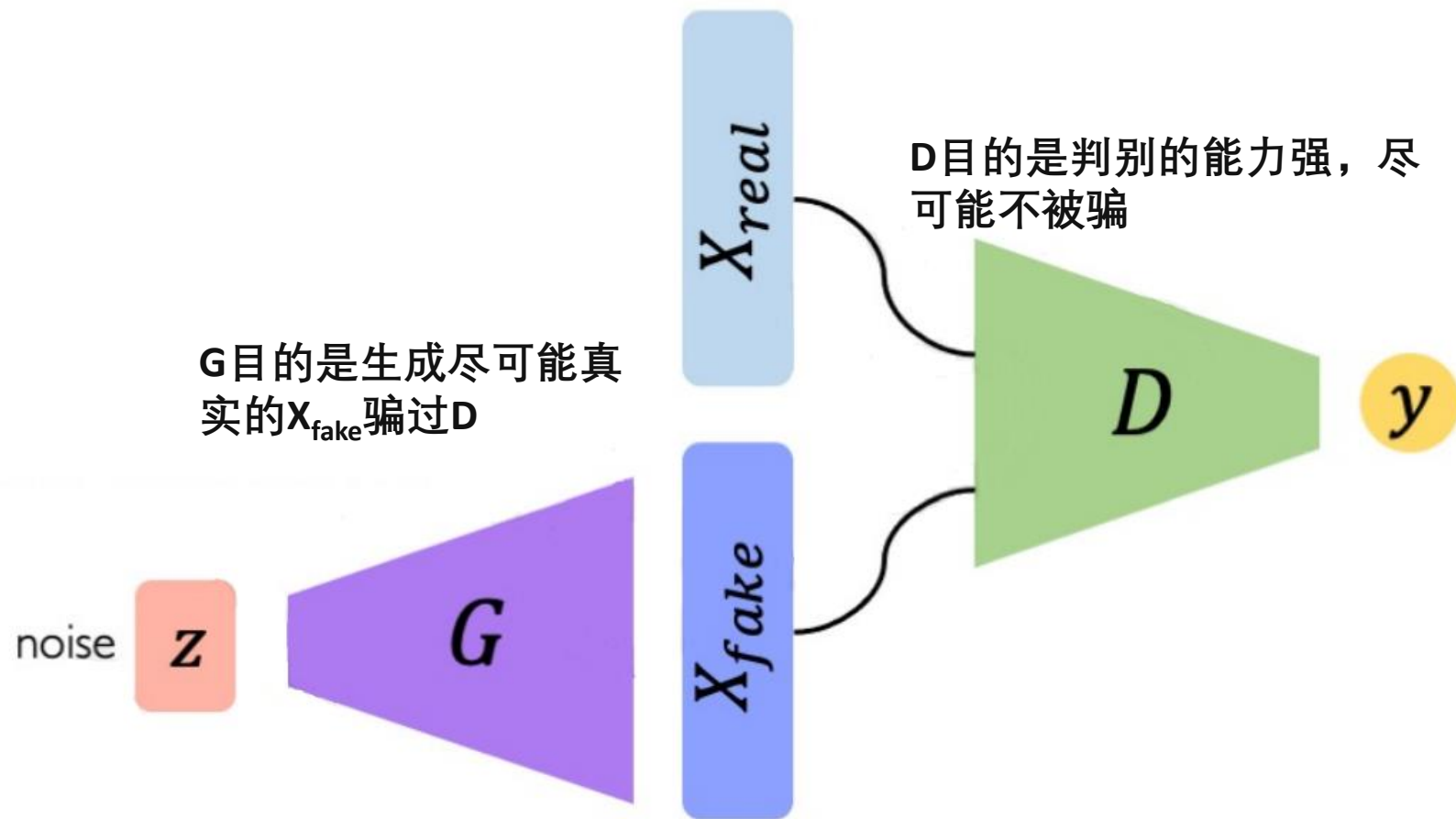
$$L_{BETA}(\phi, \beta) = -\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) + \beta D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}))$$



# 生成对抗网络(GAN)



# GAN



$$G^* = \operatorname{argmin}_G \max_D V(D, G)$$

$$V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

# GAN – 训练

思路: 固定G, 先更新D的参数, 再固定D, 更新G的参数

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by **ascending** its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by **descending** its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

GAN中有G和D, 要寻找  $G^*$

使得

$$G^* = \min_G \max_D V(G, D)$$

为了求  $G^*$ , 给定  $G$  后求出

$$\max_D V(G, D)$$

求出最优的  $D^*$ , 再给定  $D^*$  用

$$\min_G L(D^*)$$

求  $G^*$

The background of the slide features a complex, organic pattern of blue ink or smoke diffusing across a light blue background. The pattern consists of numerous fine, swirling lines and larger, more solid blue areas, creating a sense of movement and expansion. In the top-left and bottom-right corners, there are decorative elements consisting of two parallel vertical orange lines.

# **扩散模型** *(Diffusion Model)*

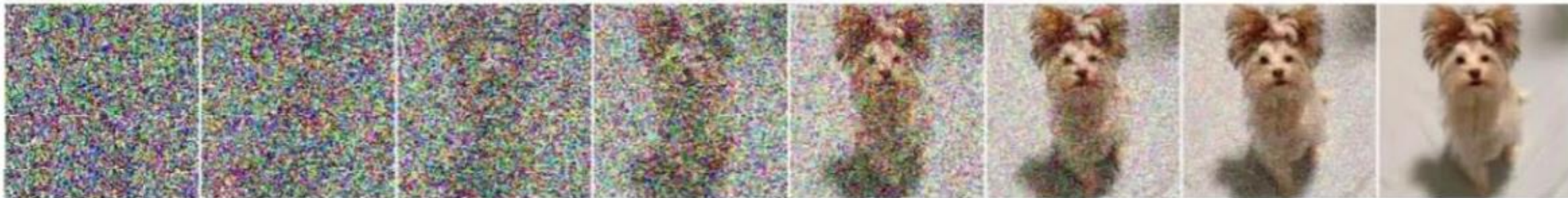


# 扩散模型

《DDPM: Denoising Diffusion Probabilistic Models》2020年  
Diffusion 的意思就是如滴入一杯清水中的墨滴一样，慢慢散开最终变成一片浑浊。  
如果这个过程可逆，那么就可以创造一个由一片浑浊去探寻最初墨滴状态的方法。

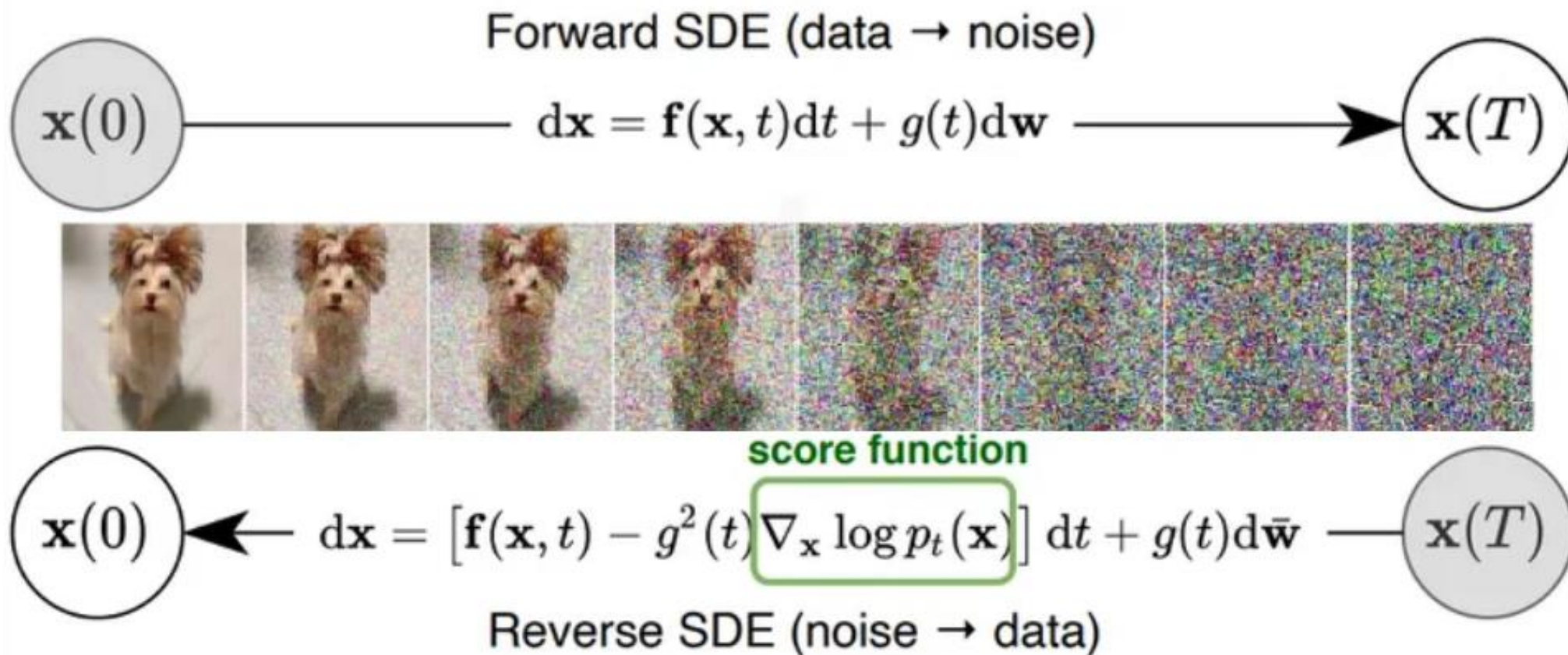


Diffusion



# 扩散模型

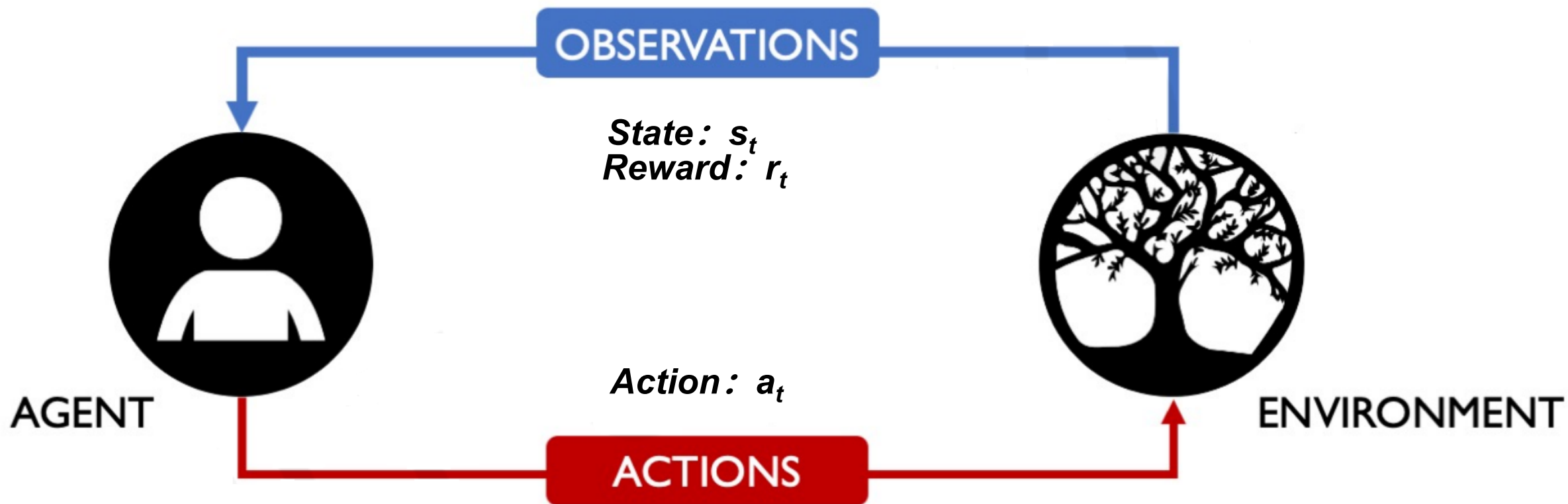
- ✓ 正向过程 (Forward Diffusion Process) 在图片中添加噪声，对图像逐步施加噪点，直至图像变成完全的高斯噪声图。这个过程用于训练阶段。
- ✓ 逆向过程 (Reverse Diffusion Process) 去除图片中的噪声，从高斯噪声逐渐还原为某一张图像。这个过程用于图像生成阶段。



2

## 深度学习与强化学习

# 强化学习



$$R_t = \sum_{i=t}^{\infty} \gamma^i r_i = \gamma^t r_t + \gamma^{t+1} r_{t+1} \dots + \gamma^{t+n} r_{t+n} + \dots$$

$\gamma$ : discount factor;  $0 < \gamma < 1$



# 强化学习

## Value Learning

Find  $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

- ✓ 输出是状态动作的值，根据值的大小选择适当的动作
- ✓ value based
- ✓ 适用于离散动作空间
- ✓ 单步更新，从一个状态，执行一个动作，到达下一个状态并获得奖励之后就进行更新

## Policy Learning

Find  $\pi(s)$

Sample  $a \sim \pi(s)$

- ✓ 输出直接是动作或者每个动作的概率，根据概率选择适当的动作
- ✓ policy based
- ✓ 适用于连续动作空间
- ✓ 回合更新，当一个动作执行完之后，获得总奖励，得到梯度，再进行参数更新

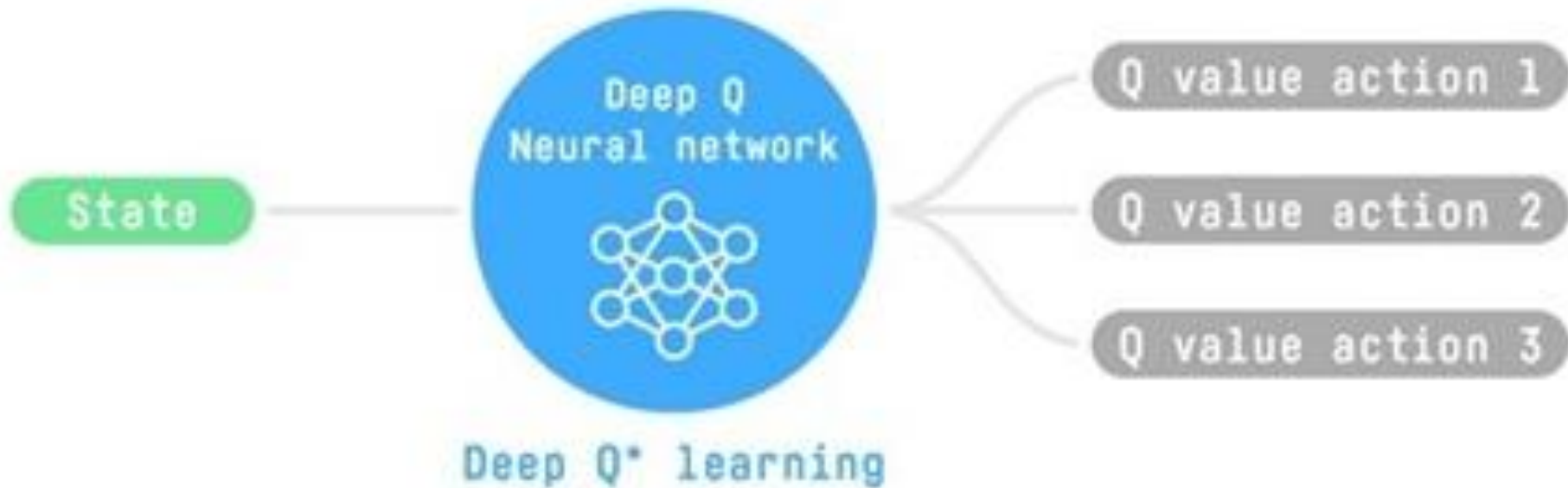
# Q-learning

Q-learning是一种value-based的强化学习方法，**训练Q函数，从而得到对应的action**

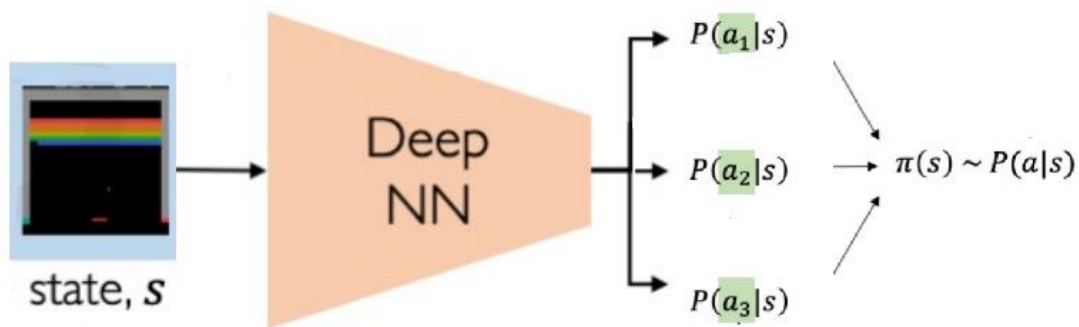


| 所有可能状态 \ 所有可能动作 | a1         | a2         | a3         |
|-----------------|------------|------------|------------|
|                 | ...        | ...        | ...        |
| s1              | $Q(s1,a1)$ | $Q(s1,a2)$ | $Q(s1,a3)$ |
| s2              | $Q(s2,a1)$ | ...        | ...        |
| s3              | $Q(s3,a1)$ | ...        | ...        |
| s4              | $Q(s4,a1)$ | ...        | $Q(s4,a3)$ |

# Q-learning



# 强化学习



**定义：**直接学习并优化策略，而无需中间步骤地估计值函数。可以是确定性策略或者随机策略，通常使用神经网络来表示策略函数。

**原理：**通过参数化策略函数来定义策略，直接优化目标函数，以最大化预期奖励。执行时，策略函数直接输出动作的概率分布。

**优势：**在连续动作空间和高维状态空间中具有优势，可以更直接地处理策略优化问题。



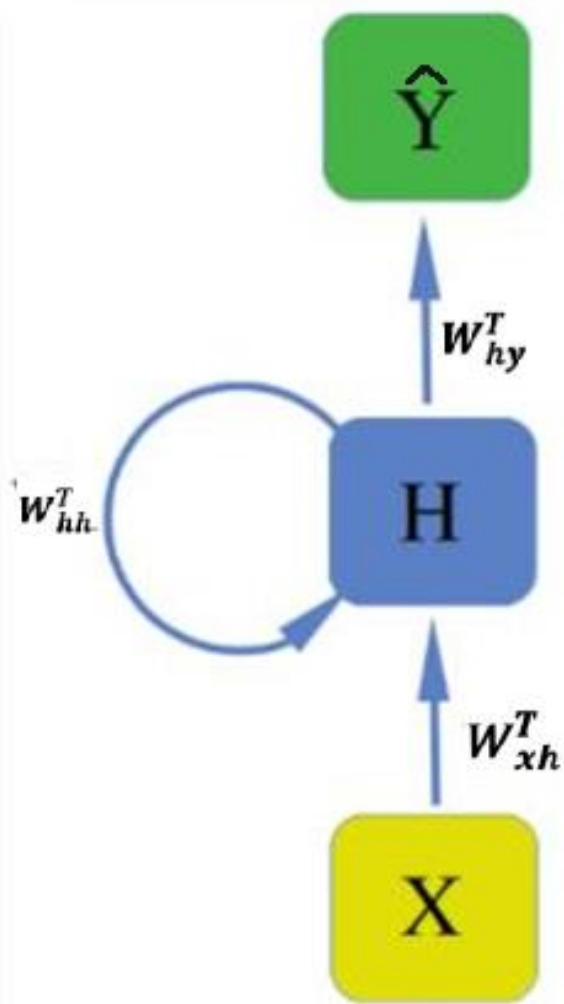
# 序列模型

RNN、LSTM

# **循环神经网络 $RNN$** ***(Recurrent Neural Network)***



# RNN



输出  $\hat{y}_t = W_{hy}^T h_t$

隐藏层的值

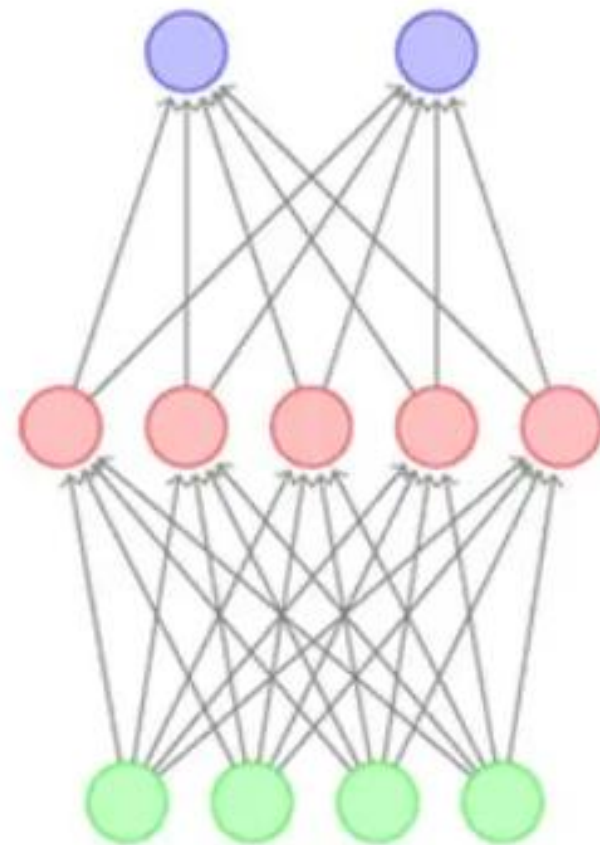
$$h_t = f(W_{hh}^T h_{t-1} + W_{xh}^T x_t)$$

输入  $x_t$

输出层

隐含层

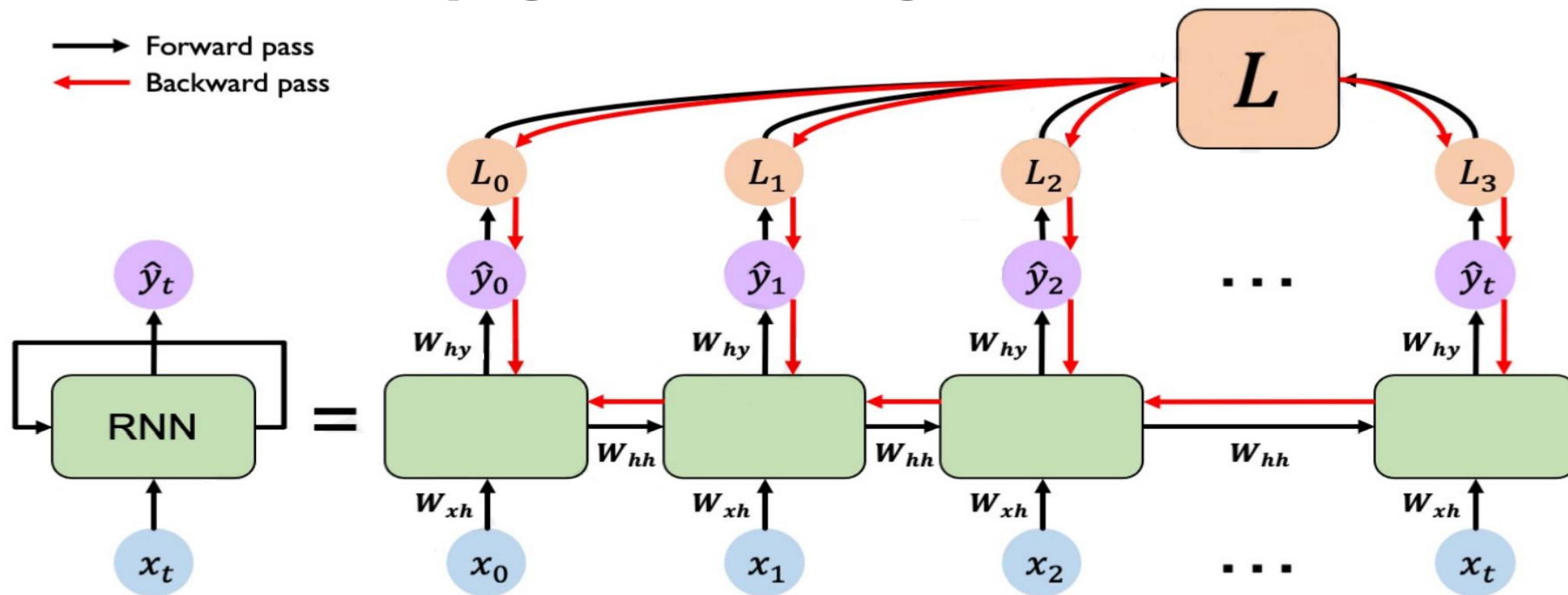
输入层



# BPTT (沿时间的反向传播)

循环神经网络使用反向传播的一种扩展方法，名为沿时间反向传播。

**BPTT ( Back-Propagation Through Time)**



$$L = \sum_{i=1}^T L_t \quad L = - \sum_{t=1}^T [\hat{y}_t \log(y_t) + (1 - \hat{y}_t) \log(1 - y_t)]$$

# BPTT的问题

梯度爆炸：设阈值

梯度消失：

- ✓ 合理的初始化权重值
- ✓ 使用不容易有软饱和现象的激活函数
- ✓ 改变网络结构

